CS 6025: Intro to Parallel Processes Assignment 3

A) Design Summary:

1. Description of adaptive integration

Adaptive Simpson's integration is a clever numerical method that focuses computational power only where it is needed most. Instead of dividing the entire range into equal pieces, the algorithm checks the curvature of the function; if the function is nearly flat, it uses a large interval, but if the function changes rapidly—like the sharp peak in your Function 1—the algorithm recursively splits that specific section into smaller and smaller sub-intervals. This ensures high accuracy without wasting time on simple, smooth areas of the curve.

2. MPI Dynamic Design

The MPI dynamic Master-Worker design is built to handle "load imbalance," which occurs when some parts of a function take much longer to calculate than others. In this architecture, a central Master (Rank 0) maintains a global stack of pending tasks, while multiple Workers (Ranks 1-N) request work whenever they become idle. When a worker encounters a complex region that requires more splitting, it sends those new sub-tasks back to the Master's stack so they can be distributed to other available workers, keeping all processors busy until the entire integral is solved.

3. Hybrid Design

The Hybrid design combines the distributed memory capabilities of MPI with the shared memory efficiency of OpenMP to maximize hardware utilization. In this model, the Master performs a static distribution, giving each MPI process a large, fixed "coarse" chunk of the total range to work on. Once a process receives its chunk, it is strictly forbidden from communicating with other processes or the master until it finishes. This approach reduces the massive communication overhead seen in your Mode 1 results by keeping the fine-grained recursive work entirely local to each process.

4. OpenMP Strategy

Inside each Hybrid worker, the OpenMP strategy utilizes Task Parallelism to manage the recursive workload across multiple CPU cores. By using the #pragma omp task directive, the program creates a new independent task every time a sub-interval needs to be split. The OpenMP runtime then automatically schedules these tasks onto the available threads within that single process. This is much faster than MPI communication because threads share the same memory space, allowing them to pass sub-intervals to one another almost instantly without the "packaging" delays that caused your speedup values to drop below 1.

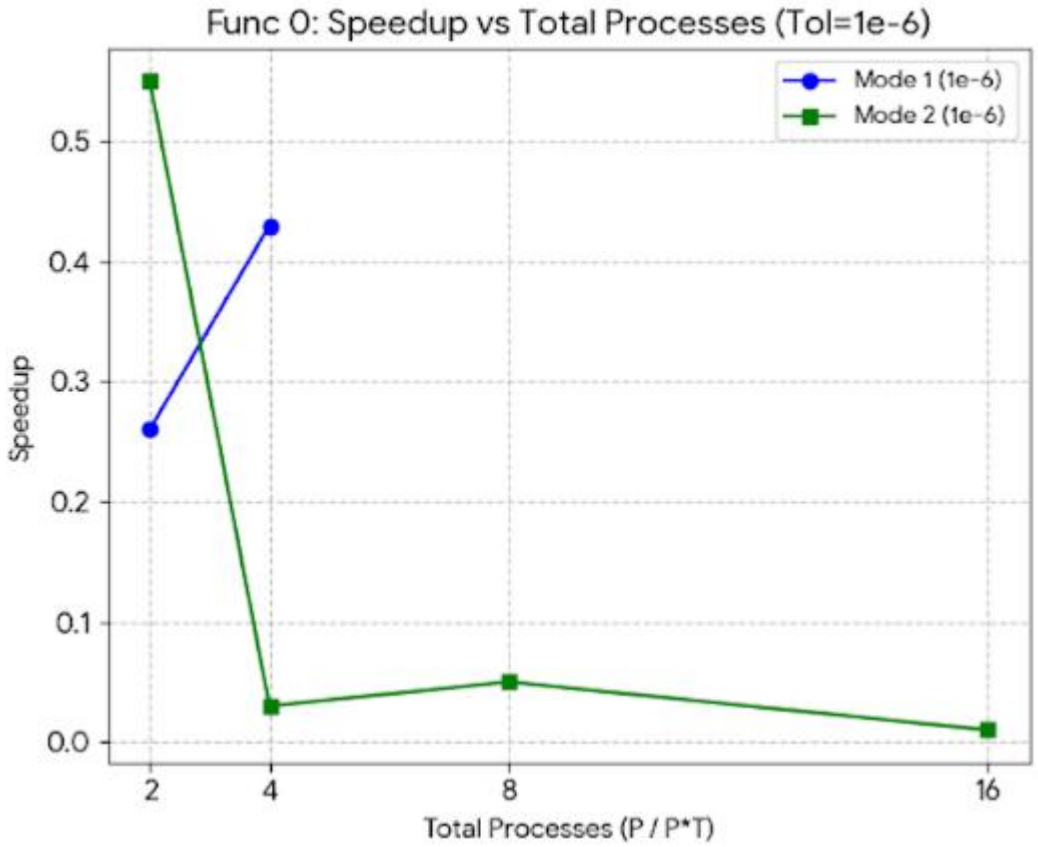
B) Result Tables:

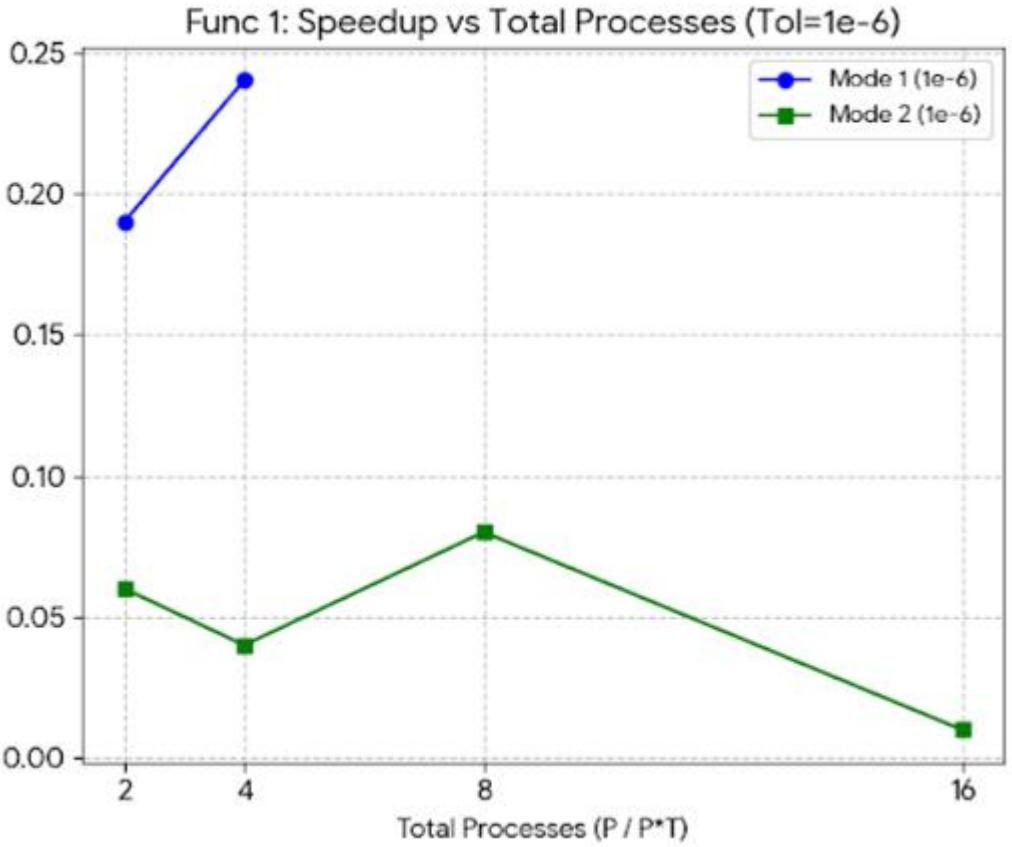
Added the excel file **A3.xlsx** along with this report in the ZIP file package. The results are porvided function wise, in separate sheets.

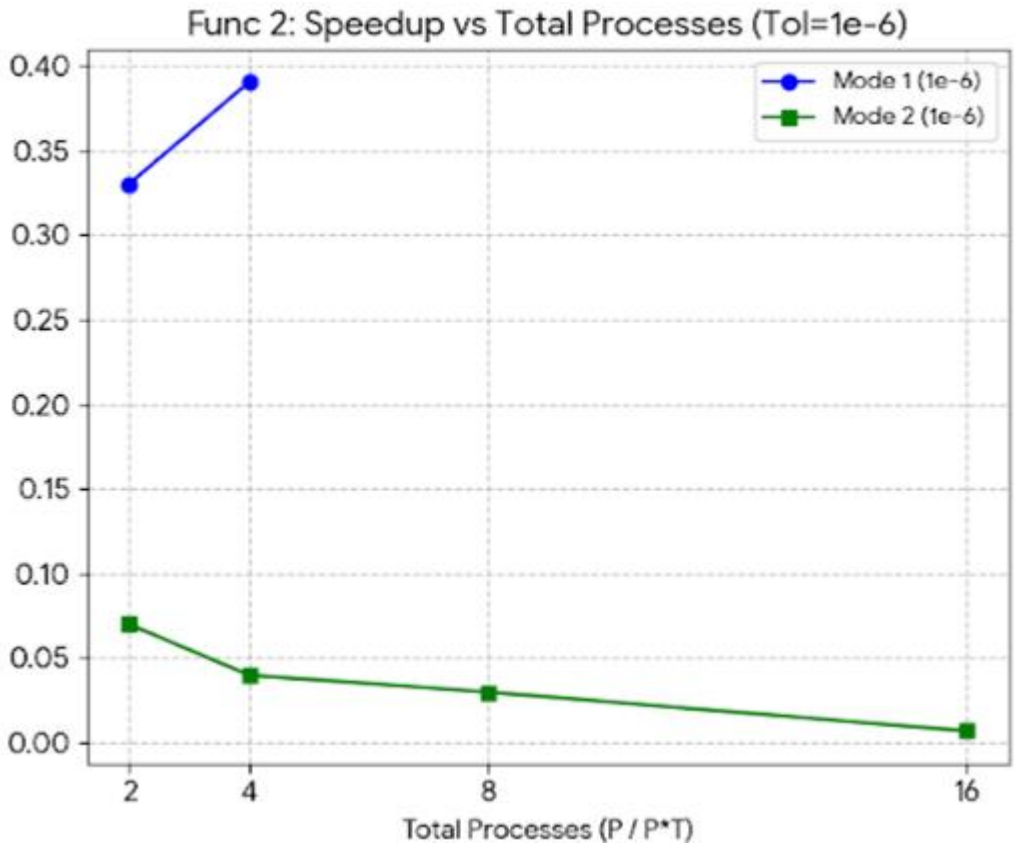
C) Plots:

1. Speedup vs Processes (for function 0, 1 and 2)

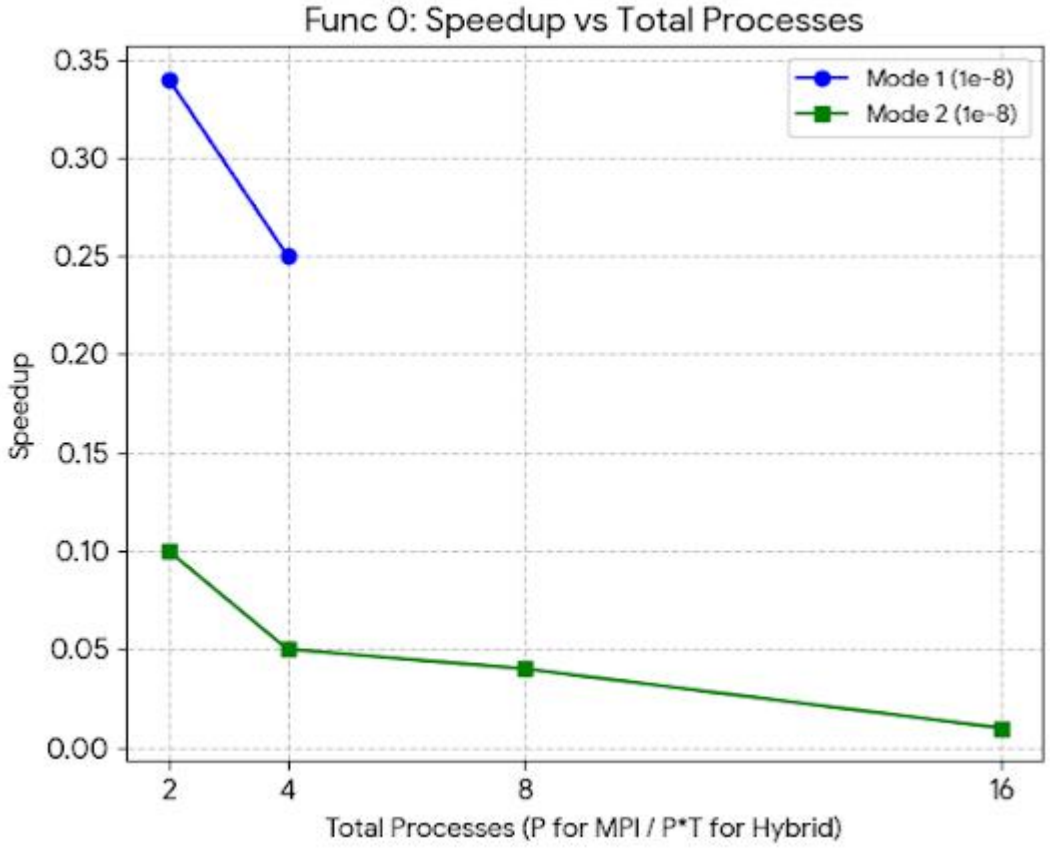
a) Error Tolerance 1e-6

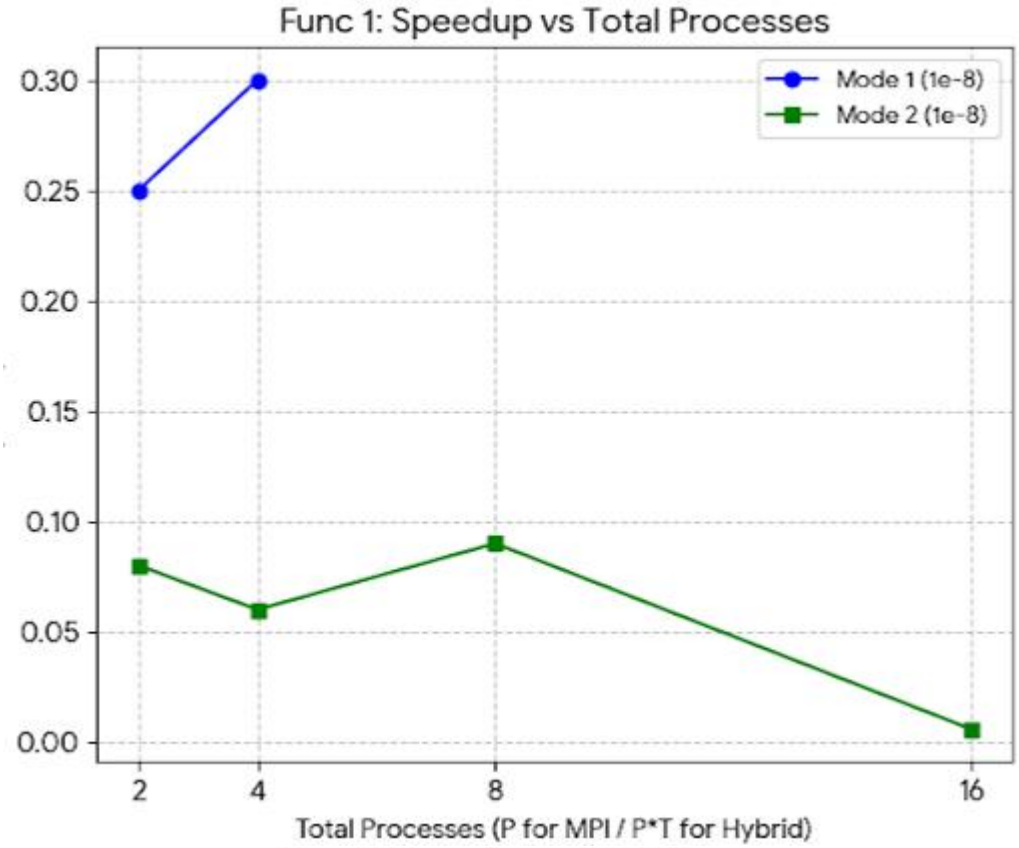

Func 0: Speedup vs Total Processes (Tol=1e-6)
Mode 1 (1e-6)
Mode 2 (1e-6)
Speedup
0.0
0.1
0.2
0.3
0.4
0.5
2
4
8
16
Total Processes (P / P*T)

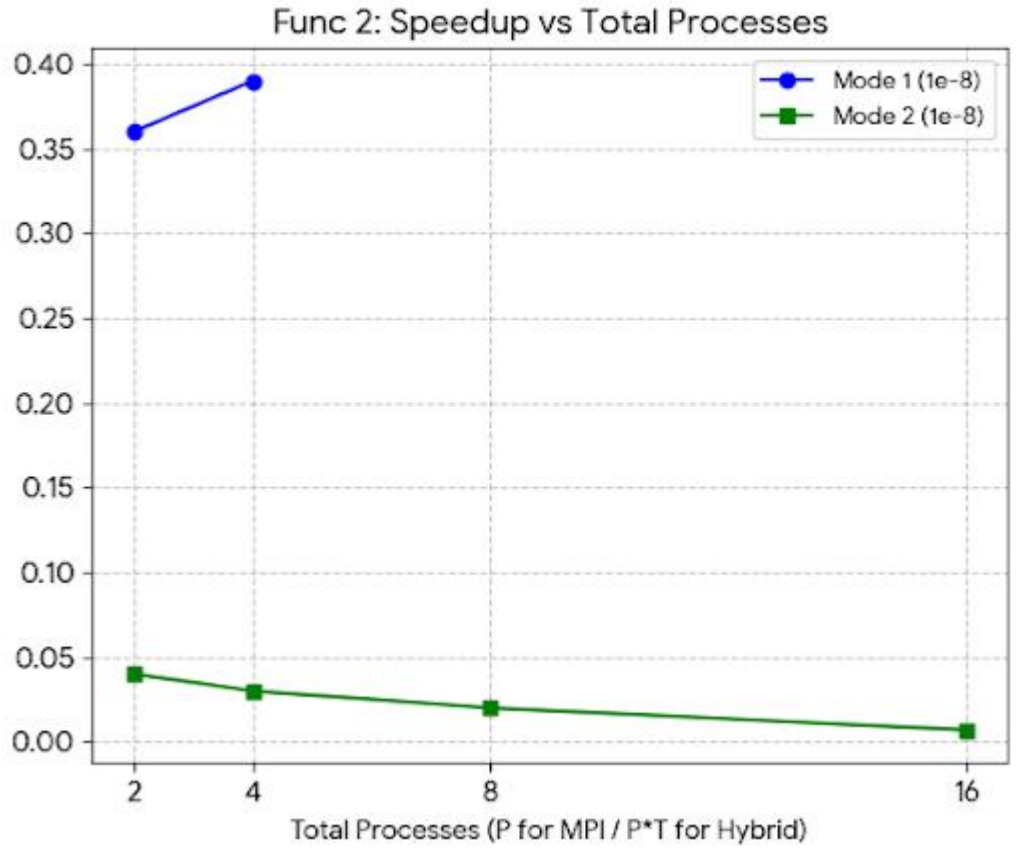

Func 1: Speedup vs Total Processes (Tol=1e-6)
Mode 1 (1e-6)
Mode 2 (1e-6)
0.00
0.05
0.10
0.15
0.20
0.25
2
4
8
16
Total Processes (P / P*T)



b) Error Tolerance 1e-8

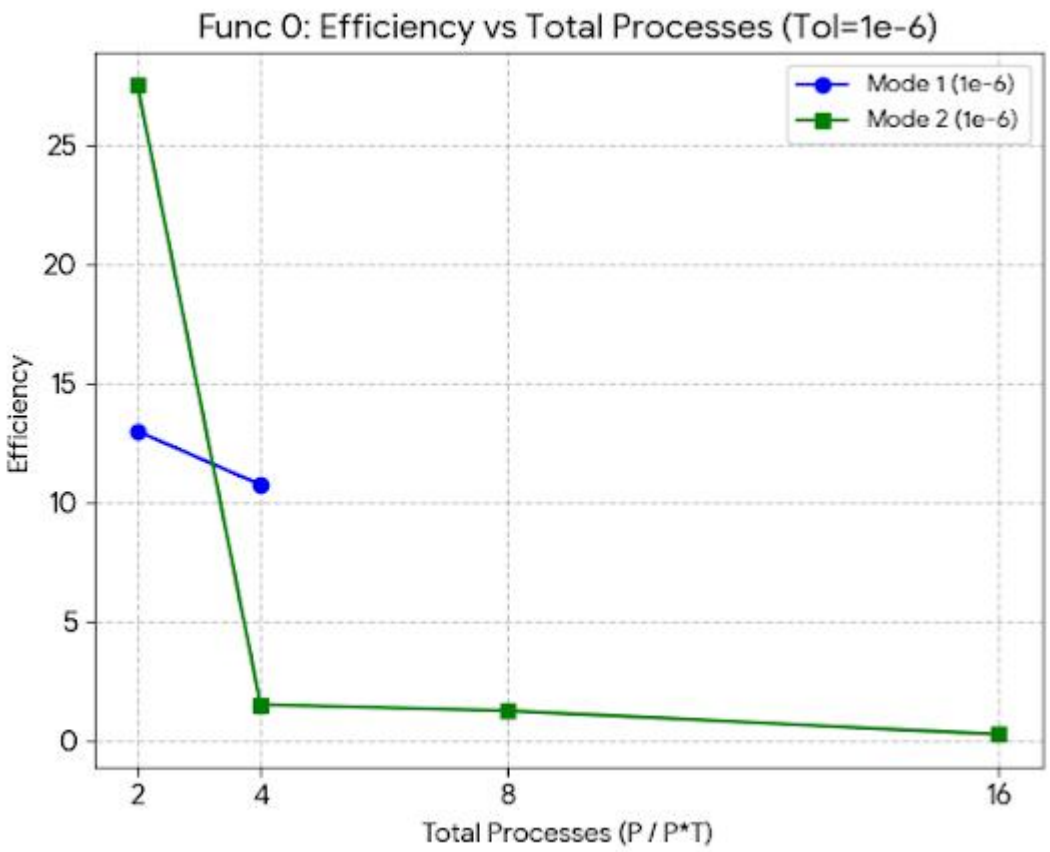


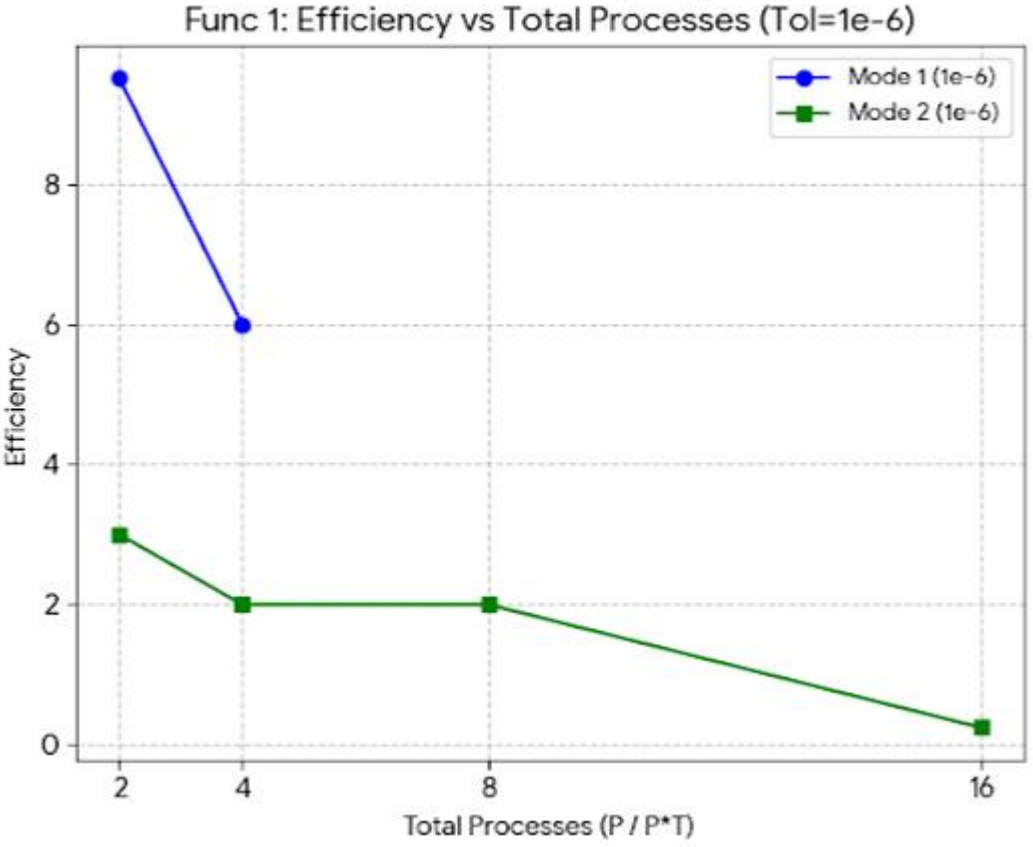


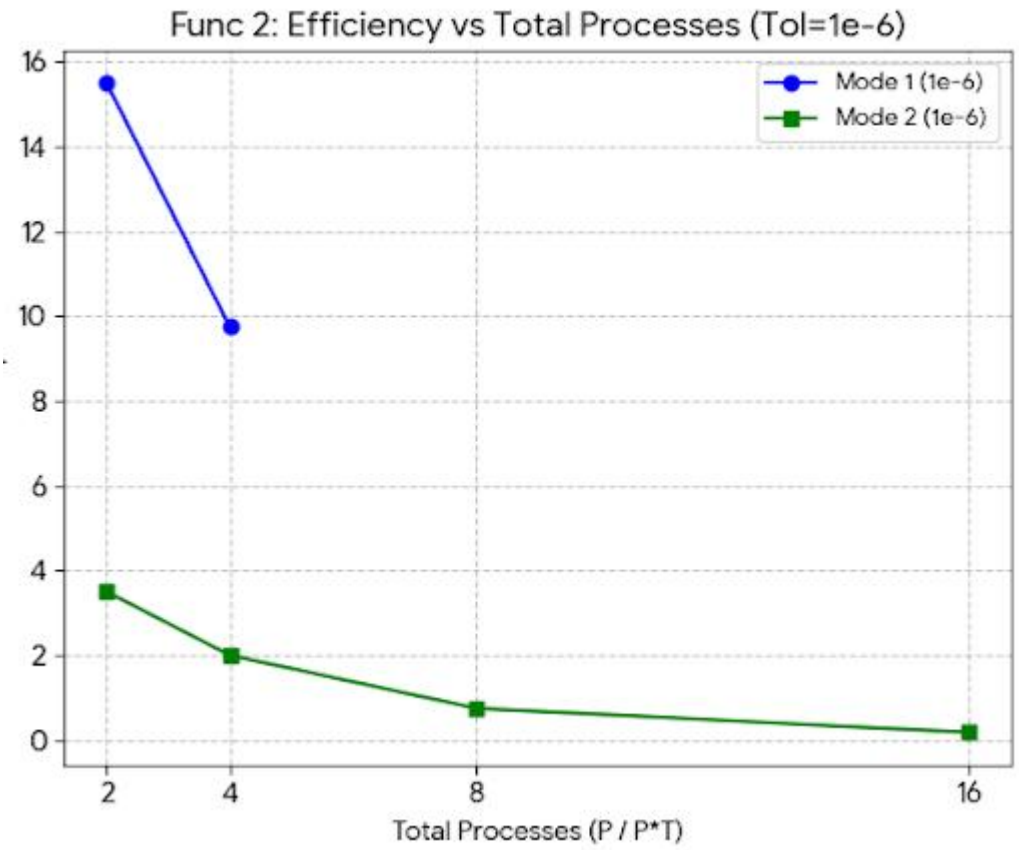


2. Efficiency vs Processes (for function 0, 1 and 2)

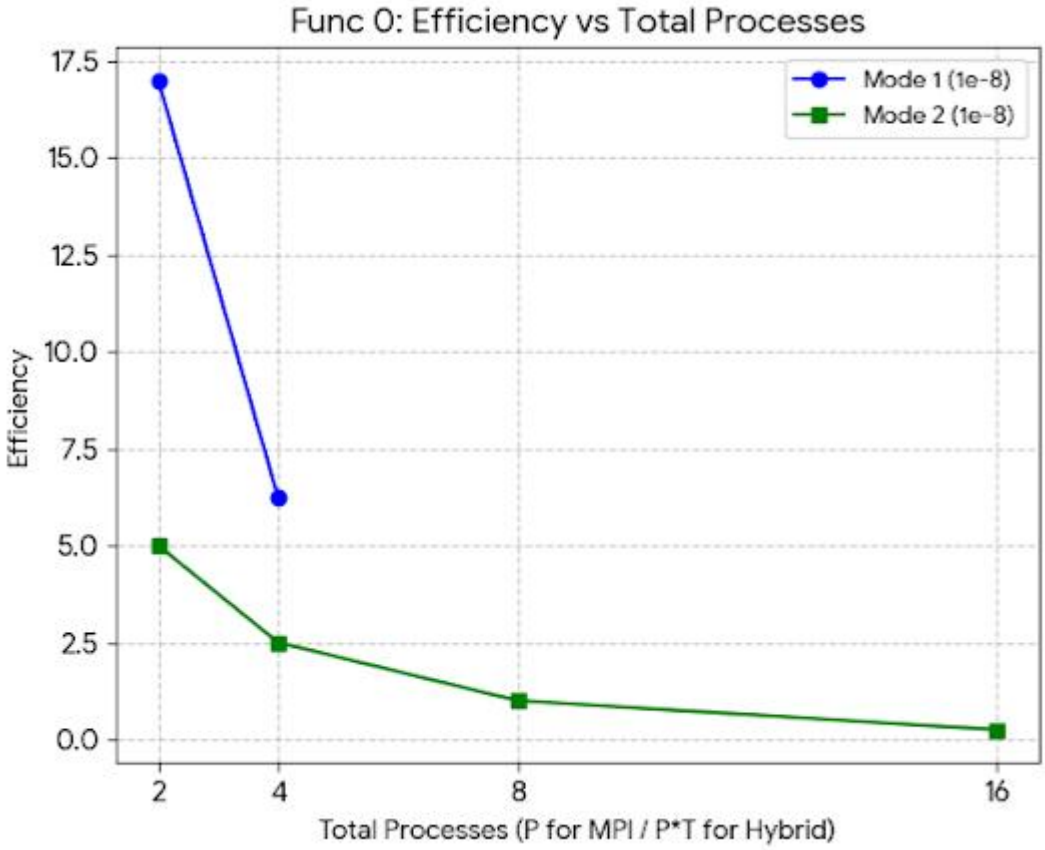
a) Error Tolerance 1e-6

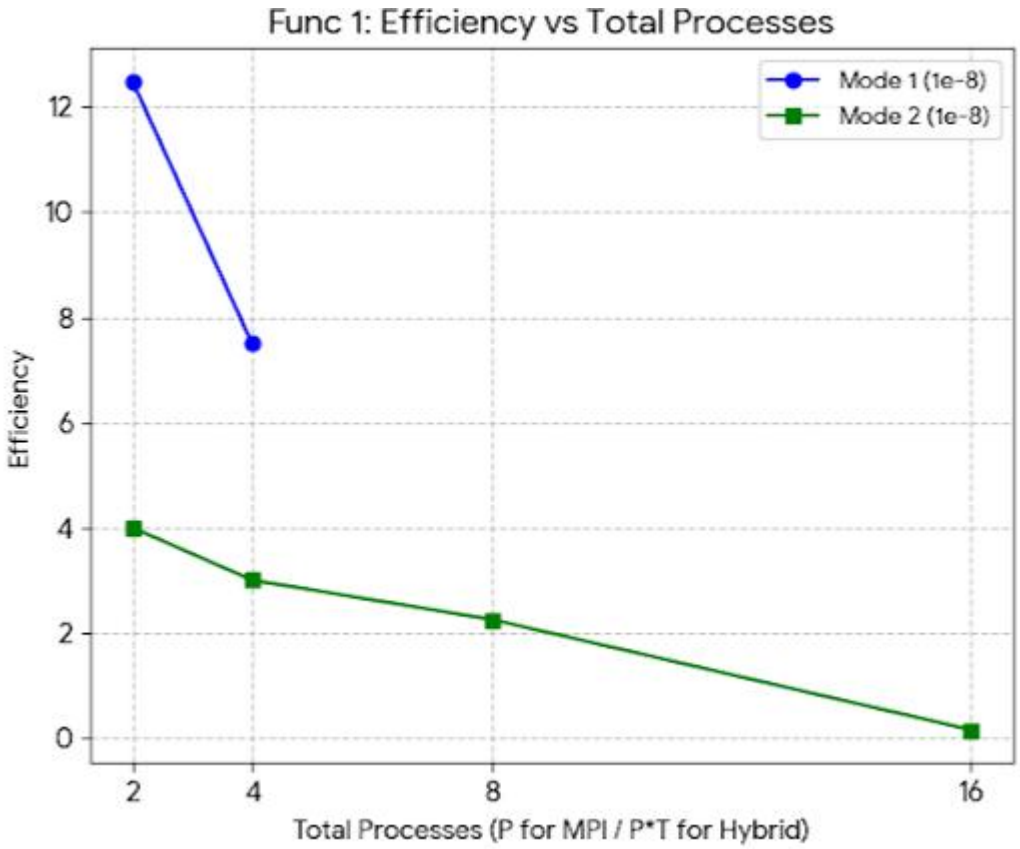

Func 0: Efficiency vs Total Processes (Tol=1e-6)
Mode 1 (1e-6)
Mode 2 (1e-6)
Efficiency
25
20
15
10
5
0
2
4
8
16
Total Processes (P / P*T)

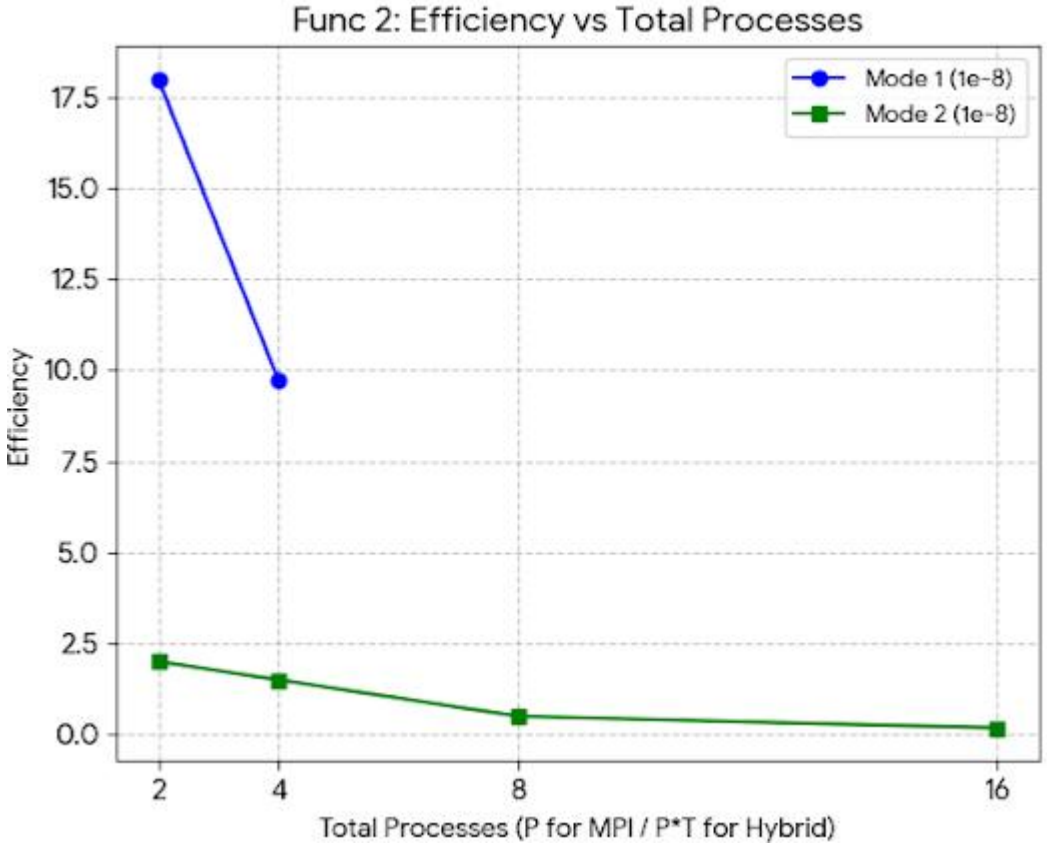

Func 1: Efficiency vs Total Processes (Tol=1e-6)
Mode 1 (1e-6)
Mode 2 (1e-6)
Efficiency
8
6
4
2
0
2
4
8
16
Total Processes (P / P*T)



b) Error Tolerance 1e-8




Func 1: Efficiency vs Total Processes
Mode 1 (1e-8)
Mode 2 (1e-8)
Efficiency
12
10
8
6
4
2
0
2
4
8
16
Total Processes (P for MPI / P*T for Hybrid)


Func 2: Efficiency vs Total Processes
Mode 1 (1e-8)
Mode 2 (1e-8)
Efficiency
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
2
4
8
16
Total Processes (P for MPI / P*T for Hybrid)

D) Discussion:

1. Why does MPI dynamic perform well for irregular workloads?

MPI dynamic design performs well for irregular workloads because it uses a "pull" model for tasks. In functions like your Function 1, which has a sharp peak in a small area, some intervals require thousands of splits while others require almost none. A dynamic master-worker system ensures that as soon as a worker finishes a simple, flat area, it can immediately request a new task from the master. This prevents a "convoy effect" where most processors sit idle waiting for one single worker to finish a high-density region, leading to much better load balancing across the entire system.

2. Why might hybrid perform worse?

Hybrid mode can perform worse primarily due to the "static" nature of how the master initially distributes work. In your implementation, the master cuts the function into fixed chunks and gives one to each MPI process. If one process is assigned a "difficult" part of the function and another gets an "easy" part, the second process will finish quickly and sit idle because there is no mechanism for it to steal work from the busy process. Additionally, the overhead of managing the OpenMP thread pool and task scheduling can actually take longer than the simple math required for the integration, especially on modern, fast CPUs.

3. Does OpenMP compensate for lack of MPI load balancing?

OpenMP does compensate for the lack of global MPI load balancing, but only **locally** within each process. While the MPI processes cannot help each other, the OpenMP threads inside a single process work together to balance the sub-intervals they were given. By using tasks, a thread that finishes its portion of the local recursion can pick up sub-tasks generated by its neighbors. This makes the individual MPI worker highly efficient at processing its assigned chunk, even if it cannot help other MPI ranks that might be struggling with their own work.

4. When does hybrid outperform MPI?

Hybrid mode typically outperforms pure MPI when the cost of communication becomes the primary bottleneck, such as when running on a cluster of multiple computers or very high-core-count servers. In a pure MPI dynamic setup, every single split requires a message to be sent to the master, which creates a "bottleneck" at the master's network interface. Hybrid mode eliminates almost all of this chatter by handling millions of small splits through shared memory threads, only using MPI for the very first and very last steps of the calculation.

5. What is the impact of thread count and process count?

The balance between thread count and process count determines the "granularity" of your parallelism. Increasing the MPI process count improves global load balancing but increases communication delays and memory usage. Increasing the OpenMP thread count reduces communication overhead and allows for faster data sharing, but it limits the system's ability to balance work across different physical nodes. On your 4-core machine, the most significant impact is oversubscription; once the total number of threads exceeds your 8 logical processors, the operating system spends more time switching between tasks than actually calculating the integral, causing the sharp performance drops seen in your results.

E) Sample Output Screenshots:

```
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/IPC Assignment 3/IPC-Assignment-3$ mpirun -np 2 ./integration 0 0 1e-6

--- Results (Mode 0) ---
Integral: 0.4832172675
Total Intervals: 13
Function ID: 0
Error Tolerance: 0.0000010000
Execution Time: 0.000017 seconds
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/IPC Assignment 3/IPC-Assignment-3$ mpirun -np 4 ./integration 0 0 1e-
8

--- Results (Mode 0) ---
Integral: 0.4832176955
Total Intervals: 45
Function ID: 0
Error Tolerance: 0.0000000100
Execution Time: 0.000035 seconds
```

```
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/IPC Assignment 3/IPC-Assignment-3$ mpirun -np 4 ./integration 1 0 1e-
8

--- Results (Mode 0) ---
Integral: 0.2677945078
Total Intervals: 141
Function ID: 1
Error Tolerance: 0.0000000100
Execution Time: 0.000076 seconds
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/IPC Assignment 3/IPC-Assignment-3$ mpirun -np 2 ./integration 1 1 1e-
8

--- Results (Mode 1) ---
Integral: 0.2677945346
Total Intervals: 63
Function ID: 1
Error Tolerance: 0.0000000100
Execution Time: 0.000429 seconds
```

```
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/IPC Assignment 3/IPC-Assignment-3$ mpirun -np 4 ./integration 2 1 1e-
6

--- Results (Mode 1) ---
Integral: -0.2586511578
Total Intervals: 3
Function ID: 2
Error Tolerance: 0.0000010000
Execution Time: 0.000046 seconds
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/IPC Assignment 3/IPC-Assignment-3$ mpirun -np 4 ./integration 2 2 1e-
6

--- Results (Mode 2) ---
Integral: 0.0041208761
Total Intervals: 230
Function ID: 2
Error Tolerance: 0.0000010000
Execution Time: 0.002723 seconds
```

```
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/IPC Assignment 3/IPC-Assignment-3$ export OMP_NUM_THREADS=2
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/IPC Assignment 3/IPC-Assignment-3$ mpirun -np 2 ./integration 0 2 1e-6

--- Results (Mode 2) ---
Integral: 0.4832176955
Total Intervals: 100
Function ID: 0
Error Tolerance: 0.0000010000
Execution Time: 0.000523 seconds
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/IPC Assignment 3/IPC-Assignment-3$ export OMP_NUM_THREADS=4
mvt@LAPTOP-UIG19S0L:/mnt/d/Assignments UNB/Semester 3/IPC/IPC Assignment 3/IPC-Assignment-3$ mpirun -np 4 ./integration 2 2 1e-8

--- Results (Mode 2) ---
Integral: 0.0041118074
Total Intervals: 754
Function ID: 2
Error Tolerance: 0.0000000100
Execution Time: 0.024721 seconds
```